

How I Did Exercise 01

On my Android Tablet / Phone

This isn't really a recommendation for how you should work on assignments in CSC 230, but it does show one option you can use if your situation is desperate. Also, it helps to show how some of the software tools we're using this semester are available on lots of different systems.

Installing Software

On Android, there's an app named Termux. It gives you a shell command prompt, along with a mechanism for installing many additional software packages you would normally expect on a Linux machine. I installed Termux and ran it. From the command prompt, I used the **pkg** command to install several different packages I thought I'd need. The commands on the left show what I ran to install the software, and the notes on the right say what I was installing:

pkg install clang	C compiler
pkg install vim	vi / vim editor
pkg install openssh	Remote login and file transfer
pkg install make	Build automation tool we'll use later
pkg install gdb	Debugging tool
pkg install valgrind	Debugging tool we'll use later

Enter the following command at the shell prompt in Termux. This will give it permissions to access files elsewhere on your Android device. You'll need this in order to access your work from the web browser once you're done.

```
$ termux-setup-storage
```

When you run this, you'll get a prompt from the system asking if you want to let Termux access photos, media and other files on your device. You should permit it to do this.

Completing the Exercise

From the Termux prompt, I made a subdirectory to work on CSC 230 assignments, then a directory under this for exercises and one more directory under that for the first exercise.

```
$ mkdir csc230  
$ cd csc230  
$ mkdir ex  
$ cd ex  
$ mkdir ex01  
$ cd ex01
```

Inside the ex01 directory, I copied down the starter files. Copy-and-paste is a little difficult on Android, but I was able to just copy the **scp** command from up near the top of the exercise into my Termux session. To paste into the Termux session, you can do a long press in the terminal session, then

choose paste from the copy-and-paste menu when it pops up.

```
$ scp unity-id@remote.eos.ncsu.edu:/mnt/coe/workspace/csc/CSC230/ex/exercise01/* .
```

Don't miss this dot
at the end.

To edit the source file, you can run **nano** in your Termux session. It looks like **nano** at least does some syntax highlighting for C, but if you'd like an editor with more capabilities, you can use **vim** instead. Be warned, an editor like **vim** takes some getting used to, so you'll want to work your way through a **vim** tutorial before you try to use it for programming.

```
$ nano arithmetic.c
```

In my editor, I added the missing lines of code, then saved the file. On my Android system, the **long** type wasn't large enough to hold the sum this program was trying to compute. I had to use **long long** instead, and I had to use the conversion specifier, **%lld** to print out the resulting sum. This let me pass the test for this program locally, but it's something that I'll need to change before I submit my solution to the exercise. On a real, common platform system, the **long** type would be sufficient for computing the sum.

After editing my source code, I exited the editor and compiled my program from the Termux prompt:

```
$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
```

If my program hadn't compiled cleanly, I'd have gone back to the editor and fixed whatever was wrong. Since it compiled OK, I'm ready to try it out. In shell commands below, I'm using output redirection to capture the program's output to a file named **output.txt**. After running the program, I check its exit status to make sure it's zero, then I compare the output against the expected output file.

```
$ ./arithmetic > output.txt  
$ echo $?  
0  
$ diff output.txt expected.txt
```

Both of these tests look good, but, before turning in my solution, I need to switch the type for the **sum** variable back to **long** along with the conversion specifier for printing the result. You can do this by bringing up your **arithmetic.c** source file in your editor.

Then, since the Android environment is not the same as the common platform, I should give it a try on a real common platform machine.

Checking on the Common Platform

From the Termux prompt, I used **sftp** to upload the source file and expected output to my NCSU Drive space. The **sftp** program is a command-line tool that lets you upload or download files from a remote system. If you're accustomed to using something like FileZilla, **sftp** does a similar job, but, instead of providing a graphical interface for transferring files, it lets you copy files back and forth by typing

commands at a prompt. You should be able to enter commands like the following to make some subdirectories in your NCSU Drive space and to copy of the needed files.

```
$ sftp unity-id@remote.eos.ncsu.edu
Connecting to remote.eos.ncsu.edu...
unity-id@remote.eos.ncsu.edu's password:
sftp> mkdir csc230
sftp> cd csc230
sftp> mkdir ex
sftp> cd ex
sftp> mkdir ex01
sftp> cd ex01
sftp> put arithmetic.c
Uploading arithmetic.c to /mnt/ncsudrive/.../ex01/arithmetic.c
sftp> put expected.txt
Uploading expected.txt to /mnt/ncsudrive.../ex01/expected.txt
sftp> exit
```

Then, I ran ssh from the Termux prompt to log in to an EOS Linux machine to try out my solution.

```
$ ssh unity-id@remote.eos.ncsu.edu
unity-id@remote.eos.ncsu.edu's password:
Last login: ... from ...
eos$ cd csc230/ex/ex01
eos$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
eos$ ./arithmetic > output.txt
eos$ echo $?
0
eos$ diff output.txt expected.txt
eos$ exit
```

Good, it worked fine the EOS Linux (after I changed the type for the sum variable and the conversion specifier used to print it).

Submitting your Solution

Since your solution worked on a common platform system, you should be ready to turn it in. As long as you didn't have to make any changes to your source code, the copy of the file you have in Termux should be in good shape. Submitting it via a web browser on your Android device is a little bit tedious. Here's what I had to do.

I went to the web browser on my device (Chrome) and navigated to the Moodle page for CSC 230. Clicking on the assignment for exercise 01 took me to a submission page where I chose "Add submission". From there, I clicked on the arrow used for dragging and dropping files into the submission. This gave me a pop-up dialog for selecting files. Over on the left, I chose "Upload a file" and then pressed the "Choose File" button. From there, I selected "Files" from the "Choose an action" panel that popped up near the bottom of the screen. That took me to a file selection interface.

From the file selector, I clicked on the hamburger icon in the upper left and scrolled down to the bottom where I was able to choose “Termux”. This gave me a view of my home directory under Termux, including the csc230 directory I made earlier. Navigating to csc230/ex/ex01 let me select the **arithmetic.c** file I was using in my Termux session. Double-clicking on this file took me back to the File picker interface in Moodle. There, I was able to choose “Upload this file” at the bottom and then “Save changes”. Then, Moodle showed that my solution to exercise 01 had been submitted for grading.

C Programming on Android

Termux isn’t a great environment for getting work done, but, I suppose if my laptop was broken, this would still let me write and debug a C program or two. I was able to run programs in **gdb**. But **valgrind** didn’t seem to give me useful reports during program execution. I was also able to run graphical programs using VNC for viewing. You can find out more at the Termux wiki: https://wiki.termux.com/wiki/Main_Page